
Cahier des Charges

Fonctionnel et Technique

Plateforme de Gestion d'Auto-École

Solution de gestion intégrée et communication temps réel

Projet	Plateforme web de gestion d'auto-école
Architecture	Microservices
Version	1.0
Date	9 juin 2026
Auteur	Ajmi

Table des matières

1	Présentation du projet	4
1.1	Contexte	4
1.2	Objectifs	4
1.3	Périmètre fonctionnel	4
1.4	Acteurs et parties prenantes	5
2	Étude de l'existant et besoins	6
2.1	Problématiques actuelles	6
2.2	Besoins exprimés	6
2.2.1	Besoins du propriétaire	6
2.2.2	Besoins du secrétariat	6
2.2.3	Besoins du moniteur	7
2.2.4	Besoins du client	7
3	Spécifications fonctionnelles	8
3.1	Module : Gestion des utilisateurs et authentification	8
3.2	Module : Gestion du parc automobile	8
3.3	Module : Planification et réservation	8
3.4	Module : Communication temps réel	9
3.5	Module : Gestion financière	9
3.6	Module : Gestion documentaire	9
3.7	Module : Notifications multicanales	9
3.8	Module : Tableau de bord analytique	10
4	Module d'apprentissage intelligent (IA)	11
4.1	Vue d'ensemble	11
4.2	Sous-module 1 : Tuteur IA conversationnel	11
4.2.1	Description fonctionnelle	11
4.2.2	Caractéristiques principales	12
4.2.3	Accès au tuteur	12
4.2.4	Confidentialité des conversations	12
4.3	Sous-module 2 : Quiz adaptatif personnalisé	13
4.3.1	Description fonctionnelle	13
4.3.2	Organisation par chapitres	13
4.3.3	Tagging des questions	13
4.3.4	Algorithme de sélection	13
4.3.5	Démarrage à froid	14
4.4	Sous-module 3 : Mode examen blanc	14

4.5	Sous-module 4 : Analyse pédagogique IA (côté gestion)	14
4.5.1	Vue par élève	14
4.5.2	Vue globale (propriétaire uniquement)	15
4.5.3	Mise en cache	15
4.6	Sous-module 5 : Notifications IA proactives	15
4.7	Sources documentaires du RAG	15
4.8	Maîtrise des coûts API	16
4.9	Architecture technique du module IA	16
4.9.1	Composants techniques	16
4.9.2	Sécurité et gouvernance	17
5	Matrice des rôles et permissions	18
6	Architecture technique	20
6.1	Vue d'ensemble	20
6.2	Stack technologique	20
6.3	Identité et authentification : Keycloak	21
6.3.1	Rôle de Keycloak	22
6.3.2	Articulation Keycloak / User Service	22
6.3.3	Sécurisation des microservices	22
6.4	Décomposition en microservices	23
6.5	Sécurité	24
7	Exigences non fonctionnelles	25
7.1	Performance	25
7.2	Disponibilité	25
7.3	Ergonomie et accessibilité	25
7.4	Maintenabilité	25
7.5	Portabilité et déploiement	26
8	Stratégie d'intégration et de déploiement continu (CI/CD)	27
8.1	Vue d'ensemble	27
8.2	Structure des dépôts et conventions	27
8.2.1	Organisation multi-dépôts	27
8.2.2	Stratégie de branches	28
8.2.3	Versionnement des artefacts	28
8.3	Pipeline d'intégration continue (CI)	28
8.4	Pipeline de déploiement continu (CD)	29
8.4.1	Environnements cibles	29
8.4.2	Stratégie de déploiement	29

8.4.3	Gestion des migrations de base de données	30
8.5	Gestion des secrets et configuration	30
8.6	Orchestration multi-dépôts	30
8.7	Phases de mise en place de la CI/CD	31
8.8	Indicateurs de qualité visés	32
9	Planning prévisionnel	33
10	Contraintes et livrables	34
10.1	Contraintes	34
10.2	Livrables attendus	34
11	Évolutions envisagées	35
12	Glossaire	36

1 Présentation du projet

1.1 Contexte

Le secteur des auto-écoles connaît une transformation digitale progressive. Les établissements gèrent encore majoritairement leurs activités à travers des outils hétérogènes : tableurs pour la planification, registres papier pour les heures de conduite, applications de messagerie grand public pour communiquer avec les élèves. Cette dispersion engendre des pertes de temps, des erreurs de planification, ainsi qu'un manque de visibilité pour la direction sur l'activité globale.

Le présent projet propose une **plateforme web unifiée** permettant de centraliser l'ensemble des opérations d'une auto-école : gestion administrative, planification des séances, communication entre moniteurs et élèves, suivi financier, et pilotage de l'activité.

1.2 Objectifs

Le projet poursuit les objectifs suivants :

- Digitaliser l'ensemble du processus métier d'une auto-école.
- Faciliter la communication temps réel entre les moniteurs et les élèves.
- Optimiser la planification des séances de conduite et de code.
- Centraliser le suivi financier et la facturation.
- Offrir un tableau de bord analytique pour la direction.
- Garantir une expérience utilisateur fluide sur web et mobile.

1.3 Périmètre fonctionnel

La plateforme couvre les domaines suivants :

- Gestion des utilisateurs et des rôles (propriétaire, secrétaire, moniteur, client).
- Gestion du parc automobile et de l'affectation aux moniteurs.
- Planification et réservation des séances de conduite et de code.
- Communication temps réel (messagerie instantanée).

-
- Gestion financière (forfaits, paiements, facturation).
 - Gestion documentaire (pièces d'identité, dossiers administratifs).
 - Notifications multicanales (Email, SMS, notifications dans l'application).
 - Reporting et tableaux de bord analytiques.
 - Module d'apprentissage intelligent du code de la route (tuteur IA, quiz adaptatif, examen blanc, analyse pédagogique).

1.4 Acteurs et parties prenantes

Acteur	Description
Propriétaire	Dirigeant de l'auto-école, dispose d'un accès complet à l'ensemble des fonctionnalités et données.
Secrétaire	Personnel administratif chargé de la gestion opérationnelle quotidienne.
Moniteur	Enseignant de la conduite, assurant les séances pratiques et théoriques.
Client (Élève)	Personne inscrite à l'auto-école pour préparer son examen du permis.

2 Étude de l'existant et besoins

2.1 Problématiques actuelles

Les auto-écoles traditionnelles font face à plusieurs difficultés :

- **Planification manuelle** : conflits d'horaires fréquents, double réservation de véhicules ou de moniteurs.
- **Communication fragmentée** : utilisation de canaux multiples (téléphone, SMS, WhatsApp) sans traçabilité.
- **Suivi financier opaque** : difficulté à connaître en temps réel le chiffre d'affaires, les impayés, les heures réalisées par moniteur.
- **Gestion documentaire papier** : risque de perte, accès difficile aux dossiers des élèves.
- **Manque d'indicateurs** : absence de tableau de bord pour piloter l'activité.

2.2 Besoins exprimés

2.2.1 Besoins du propriétaire

- Vue d'ensemble de l'activité (CA, taux d'occupation, taux de réussite aux examens).
- Gestion des comptes utilisateurs et des permissions.
- Pilotage des moniteurs, secrétaires, véhicules et finances.
- Accès à l'ensemble des rapports et statistiques.

2.2.2 Besoins du secrétariat

- Gestion du parc automobile (ajout, entretien, disponibilité).
- Gestion financière : encaissements, facturation, suivi des forfaits.
- Affectation des heures de travail aux moniteurs.
- Inscription des nouveaux clients et gestion de leur dossier.

2.2.3 Besoins du moniteur

- Consultation du calendrier journalier et hebdomadaire.
- Communication directe avec les clients (messagerie).
- Validation ou refus des demandes de séance.
- Suivi de la progression pédagogique de chaque élève.

2.2.4 Besoins du client

- Réservation en ligne de séances de conduite ou de code.
- Communication avec son moniteur.
- Consultation de son planning, de son forfait et de sa progression.
- Accès à son historique de paiements et à ses documents.

3 Spécifications fonctionnelles

3.1 Module : Gestion des utilisateurs et authentification

- Inscription et création de compte (client en libre-service, autres rôles créés par le propriétaire ou la secrétaire).
- Authentification sécurisée (JWT + Refresh Token).
- Gestion des rôles et permissions (RBAC — Role-Based Access Control).
- Réinitialisation de mot de passe par email.
- Gestion du profil (informations personnelles, photo, préférences).

3.2 Module : Gestion du parc automobile

- Enregistrement des véhicules (marque, modèle, immatriculation, type de boîte, carburant).
- Suivi de l'entretien (révisions, contrôle technique, assurance).
- Affectation d'un véhicule à un moniteur ou à une séance.
- État de disponibilité en temps réel.
- Historique d'utilisation par véhicule.

3.3 Module : Planification et réservation

- Calendrier interactif (vue jour, semaine, mois).
- Réservation de séances par le client (conduite ou code).
- Système anti-conflit : détection des chevauchements (moniteur, véhicule, élève).
- Validation manuelle ou automatique selon paramétrage.
- Annulation et report de séances avec règles de préavis.
- Affectation automatique d'un véhicule disponible.

3.4 Module : Communication temps réel

- Messagerie instantanée entre moniteur et client via WebSocket.
- Indicateur de présence en ligne et accusé de lecture.
- Historique des conversations.
- Envoi de fichiers (photos, documents).
- Notifications push lors de nouveaux messages.

3.5 Module : Gestion financière

- Définition des forfaits (heures de conduite, sessions de code, prix).
- Inscription d'un client à un forfait.
- Suivi de la consommation (heures réalisées vs. heures du forfait).
- Enregistrement des paiements (espèces, chèque, virement, carte).
- Échéanciers de paiement.
- Génération de factures et reçus en PDF.
- Suivi des impayés et relances.

3.6 Module : Gestion documentaire

- Upload et stockage sécurisé des documents (CIN, photo, dossier d'inscription).
- Classement par dossier client.
- Téléchargement et consultation par les rôles autorisés.
- Suivi de la validité des documents (date d'expiration).

3.7 Module : Notifications multicanales

- Notifications in-app (cloche, badges).
- Notifications par Email (confirmation de réservation, rappel de séance, facture).
- Notifications par SMS (rappel 24h avant la séance, alerte d'annulation).

- Configuration des préférences de notification par utilisateur.

3.8 Module : Tableau de bord analytique

- Indicateurs financiers : CA mensuel/annuel, encaissements, impayés.
- Indicateurs opérationnels : taux d'occupation des moniteurs, taux d'utilisation des véhicules.
- Indicateurs pédagogiques : taux de réussite à l'examen, durée moyenne de formation.
- Visualisations graphiques (courbes, histogrammes, jauges).
- Export des données (PDF, Excel).

4 Module d'apprentissage intelligent (IA)

4.1 Vue d'ensemble

La plateforme intègre un **module d'intelligence artificielle** dédié à l'apprentissage du code de la route. Ce module ne remplace pas l'enseignement humain : il vient en **complément pédagogique** pour offrir à l'élève un accompagnement disponible 24h/24, et à l'auto-école une vision fine de la progression de chaque apprenant.

Le module est construit autour d'une architecture **RAG (Retrieval-Augmented Generation)**, qui combine la puissance d'un grand modèle de langage (LLM) avec une base documentaire propre à l'auto-école. Cette approche garantit des réponses ancrées dans les sources officielles et limite le risque d'erreur ou d'hallucination.

Trois fonctionnalités IA, trois publics

Tuteur conversationnel pour l'élève — explication des réponses et chat libre sur le code.

Quiz adaptatif pour l'élève — entraînement personnalisé ciblant les points faibles.

Analyse pédagogique pour le propriétaire et la secrétaire — synthèse intelligente des performances de chaque élève.

4.2 Sous-module 1 : Tuteur IA conversationnel

4.2.1 Description fonctionnelle

Le tuteur IA est un assistant conversationnel accessible à l'élève à tout moment. Il intervient dans deux contextes :

- **Explication contextuelle après quiz** — Lorsqu'une question est ratée, un bouton « Pourquoi ? » permet d'obtenir une explication détaillée de la bonne réponse, illustrée par la règle correspondante du code de la route.
- **Chat libre** — L'élève peut poser n'importe quelle question sur le code (panneaux, priorités, vitesse, situations particulières) via une interface de messagerie dédiée.

L'élève peut enchaîner des questions de suivi pour approfondir un sujet (« *Et si le feu était orange clignotant ?* », « *Cette règle s'applique-t-elle hors agglomération ?* »).

4.2.2 Caractéristiques principales

- Disponibilité bilingue : **français et arabe**. L'élève choisit sa langue de préférence dans son profil et peut la changer en cours de conversation.
- Réponses streamées en temps réel (effet « machine à écrire »).
- Chaque réponse est accompagnée de la **source documentaire** citée (extrait du code de la route ou du manuel).
- Affichage systématique d'un disclaimer : « *Réponse générée par IA. En cas de doute, consultez votre moniteur.* »
- Bouton de feedback « Cette réponse est-elle utile ? » (pouce haut/bas) pour signaler les réponses problématiques.
- Mécanisme de **fallback sécurisé** : si le système ne trouve pas de source pertinente dans la base documentaire, il répond « *Je ne dispose pas d'information fiable sur ce point, contactez votre auto-école* » plutôt que d'inventer.

4.2.3 Accès au tuteur

L'usage du tuteur conversationnel est ouvert à trois rôles :

- **Client** — usage principal, dans le cadre de son apprentissage.
- **Secrétaire** — consultation possible, notamment pour répondre aux questions des clients par téléphone ou au comptoir.
- **Propriétaire** — accès complet pour vérification et supervision.

Le **Moniteur** n'a pas accès au tuteur IA : son rôle est centré sur la pratique de la conduite, et l'enseignement théorique est porté par la secrétaire et le tuteur IA.

4.2.4 Confidentialité des conversations

L'historique des conversations d'un élève avec le tuteur est **strictement confidentiel** : seul l'élève peut consulter ses échanges passés. Ni le propriétaire, ni la secrétaire, ni le moniteur n'ont accès au contenu des conversations d'un élève. Cette règle garantit un espace d'apprentissage rassurant où l'élève peut poser librement toutes ses questions, y compris les plus basiques.

4.3 Sous-module 2 : Quiz adaptatif personnalisé

4.3.1 Description fonctionnelle

Plutôt que de servir des questions aléatoires, le système analyse les performances de chaque élève et compose des sessions de révision ciblant prioritairement ses **points faibles**.

4.3.2 Organisation par chapitres

Le corpus de questions est structuré par **chapitres thématiques** (signalisation, priorités, vitesse, stationnement, conduite défensive, premiers secours, etc.). L'élève peut :

- Réviser **un chapitre spécifique** (mode focus).
- Choisir le **mode mixte** : sélection de plusieurs chapitres, ou ensemble du corpus, avec pondération adaptative.

4.3.3 Tagging des questions

Lorsque la secrétaire ou le propriétaire uploadé un lot de questions :

- L'IA **classifie automatiquement** chaque question dans le chapitre approprié (tagging automatique par analyse sémantique).
- La secrétaire peut **corriger manuellement** les tags proposés ou en ajouter (interface de revue avant publication).
- Une question peut être associée à **plusieurs chapitres** (multi-tag) si elle couvre plusieurs thèmes.

4.3.4 Algorithme de sélection

Pour chaque élève, le système maintient un **score de maîtrise** par chapitre (valeur entre 0 et 1). La composition du prochain quiz suit la règle suivante :

- 60% de questions issues des chapitres faibles (score < 0,5).
- 30% de questions issues des chapitres moyens (score entre 0,5 et 0,8).
- 10% de questions nouvelles ou récemment échouées, en logique de **répétition espacée** (réapparition à J+1, J+3, J+7).

4.3.5 Démarrage à froid

Lorsqu'un nouvel élève s'inscrit, le système n'a aucune donnée sur ses connaissances. Un **quiz initial de calibration** de 20 questions, couvrant tous les chapitres de manière équilibrée, est proposé à la première connexion. Les résultats initialisent les scores de maîtrise par chapitre.

4.4 Sous-module 3 : Mode examen blanc

Ce mode simule les conditions réelles de l'examen officiel :

- Nombre de questions fixe (paramétrable par l'auto-école, typiquement 40 questions).
- Durée limitée et chronomètre visible.
- Aucune assistance IA pendant l'examen (le bouton « Pourquoi ? » et le chat sont désactivés).
- Note finale et seuil de réussite affichés à la soumission.
- Correction IA détaillée disponible **après** la fin de l'examen, question par question.
- Statistiques d'examens blancs trackées séparément des sessions de révision.

4.5 Sous-module 4 : Analyse pédagogique IA (côté gestion)

4.5.1 Vue par élève

Le propriétaire et la secrétaire disposent d'un tableau de bord IA pour chaque élève, comprenant :

- Une **synthèse en langage naturel** générée à la demande, par exemple : « *Mariam a réalisé 450 questions ce mois-ci avec un taux de réussite global de 72%. Forte progression sur la signalisation (+25%), mais faiblesse persistante sur les priorités à droite (38% de réussite). Recommandation : prévoir une session de révision ciblée avant l'examen blanc prévu le 15.* »
- **Indice de prédiction** de réussite à l'examen, calculé à partir des scores actuels et de l'évolution récente.
- **Recommandations actionnables** : élèves prêts pour l'examen, élèves en stagnation, élèves à recontacter.

4.5.2 Vue globale (propriétaire uniquement)

- Taux de réussite moyen aux examens blancs, par mois et par promotion.
- Identification des chapitres globalement les plus difficiles pour les élèves de l'auto-école.
- Suggestions de **cours de groupe** ou de supports pédagogiques à renforcer.

4.5.3 Mise en cache

Les synthèses IA sont mises en cache pendant 24 heures pour limiter le coût des appels au LLM tout en conservant une information à jour quotidiennement.

4.6 Sous-module 5 : Notifications IA proactives

Le système initie des notifications intelligentes (envoyées via le module Notifications) :

- **Rappel d'inactivité** — « *Tu n'as pas révisé depuis 5 jours, voici 10 questions ciblées sur tes points faibles.* »
- **Recommandation d'examen blanc** — « *Ton niveau atteint le seuil de réussite, tu peux passer un examen blanc.* »
- **Encouragement après progression** — « *Ton score sur la signalisation est passé de 45% à 78%, bravo !* »
- **Alerte de stagnation** (à destination de la secrétaire) — « *3 élèves stagnent depuis plus de 10 jours, un suivi serait bénéfique.* »

L'élève peut désactiver les notifications IA dans ses préférences.

4.7 Sources documentaires du RAG

La base documentaire indexée alimente toutes les fonctionnalités IA. Quatre catégories de sources sont prévues :

- **Code de la route tunisien officiel** — documents PDF officiels (français et arabe).
- **Manuel de l'auto-école** — support pédagogique propre à l'établissement, s'il en dispose.
- **FAQ rédigée par les moniteurs et la direction** — réponses aux questions fréquentes des élèves, capitalisation du savoir interne.

- **Banque de questions avec explications** — les questions de quiz uploadées sont elles-mêmes indexées avec leurs corrigés détaillés.

L'ingestion d'un nouveau document déclenche automatiquement le chunking (découpage en passages), le calcul des embeddings et l'indexation dans la base vectorielle.

4.8 Maîtrise des coûts API

L'utilisation d'un LLM via API externe (OpenAI ou Anthropic) génère un coût à l'usage. Le système intègre les mécanismes suivants :

- **Budget mensuel global** configurable par le propriétaire (plafond en dinars / euros).
- Suivi en temps réel de la consommation et alerte à 80% du budget.
- Suspension automatique des fonctionnalités IA si le budget est dépassé, avec notification au propriétaire.
- **Tableau de bord des coûts IA** réservé au propriétaire, affichant :
 - Coût total du mois en cours, par jour et par fonctionnalité (tuteur, quiz, analyse).
 - Coût moyen par élève actif.
 - Top 10 des élèves consommateurs (pour détecter d'éventuels abus).
 - Projection de coût en fin de mois.
- Mise en cache agressive des synthèses pédagogiques (24h) et des réponses aux questions fréquentes.

4.9 Architecture technique du module IA

Microservice dédié : ai-service

Le module IA est porté par un **microservice dédié (ai-service)**, isolé du reste de la plateforme pour des raisons de sécurité, de scalabilité et de maîtrise des coûts.

4.9.1 Composants techniques

Composant	Rôle
LLM (API externe)	OpenAI GPT-4o ou Anthropic Claude. Provider abstrait par interface pour permettre le changement ou l'ajout d'alternatives.
Base vectorielle	Qdrant — stockage des embeddings et recherche de similarité.
Modèle d'embedding	text-embedding-3-small (OpenAI) ou équivalent open-source.
Orchestration RAG	LangChain (Java/Python via interop) ou Spring AI.
Pipeline d'ingestion	Service de chunking et d'indexation déclenché à l'upload d'un document.
Cache des réponses	Redis — mise en cache des synthèses et des questions fréquentes.
Stockage des conversations	PostgreSQL — historique par élève (cloisonné par utilisateur).
Streaming temps réel	WebSocket — réponses du tuteur diffusées token par token.

4.9.2 Sécurité et gouvernance

- Anonymisation des données envoyées à l'API LLM externe (pas de nom complet, pas d'identifiant personnel).
- Conformité avec les bonnes pratiques de protection des données personnelles.
- Logs d'audit de toutes les requêtes IA (à des fins de qualité et de coût, pas pour consultation par les autres rôles).
- Possibilité de désactiver complètement le module IA depuis la configuration du propriétaire.

5 Matrice des rôles et permissions

Fonctionnalité	Propriétaire	Secrétaire	Moniteur	Client
Gestion des utilisateurs	✓	✓	—	—
Gestion du parc automobile	✓	✓	Lecture	—
Gestion financière	✓	✓	—	Lecture (perso)
Affectation des heures	✓	✓	—	—
Planification générale	✓	✓	Lecture	—
Calendrier personnel	✓	✓	✓	✓
Réservation de séance	✓	✓	—	✓
Validation des demandes	✓	✓	✓	—
Messagerie	✓	✓	✓	✓
Tableau de bord global	✓	Limité	—	—
Gestion documentaire	✓	✓	Lecture	Lecture (perso)
Rapports et exports	✓	✓	—	—
<i>Fonctionnalités IA</i>				
Upload & tagging des questions	✓	✓	—	—
Tuteur IA conversationnel	✓	✓	—	✓
Quiz adaptatif	—	—	—	✓
Examen blanc	—	—	—	✓
Historique chat IA (perso)	—	—	—	✓
Analyse pédagogique par élève	✓	✓	—	—
Analyse pédagogique globale	✓	—	—	—

Fonctionnalité	Propriétaire	Secrétaire	Moniteur	Client
Dashboard coûts IA	✓	—	—	—
Configuration IA (budget, prompts)	✓	—	—	—

6 Architecture technique

6.1 Vue d'ensemble

L'application repose sur une **architecture microservices**, permettant une séparation claire des responsabilités, un déploiement indépendant des services, et une scalabilité horizontale. Chaque microservice gère un domaine métier spécifique et expose ses fonctionnalités via une API REST sécurisée.

6.2 Stack technologique

Composant	Technologie
Frontend	Angular (TypeScript)
Backend	Java 17+ avec Spring Boot
Framework micro-services	Spring Cloud (Gateway, Config, Discovery)
API Gateway	Spring Cloud Gateway
Service Discovery	Netflix Eureka (ou Consul)
Communication inter-services	REST (synchrone) + RabbitMQ ou Kafka (asynchrone)
Communication temps réel	WebSocket (STOMP / SockJS)
Base de données	PostgreSQL (une instance par microservice)
Cache	Redis

Technologie	
Composant	
Stockage de fichiers	MinIO (S3-compatible)
Base vectorielle (IA)	Qdrant — stockage et recherche d’embeddings pour le RAG
LLM (IA)	OpenAI GPT-4o ou Anthropic Claude (API externe, provider abstrait)
Orchestration IA	LangChain ou Spring AI
Authentification & IAM	Keycloak (Identity Provider) + Spring Security OAuth2 Resource Server
Conteneurisation	Docker + Docker Compose
Orchestration (optionnel)	Kubernetes
Monitoring	Prometheus + Grafana
Journalisation	ELK Stack (Elasticsearch, Logstash, Kibana)
CI/CD	GitLab CI ou Jenkins

6.3 Identité et authentification : Keycloak

L’authentification, la gestion des utilisateurs et l’émission des tokens JWT sont déléguées à **Keycloak**, une solution open-source d’Identity and Access Management (IAM) éprouvée en production.

Pourquoi Keycloak plutôt qu'un service maison

Keycloak est un **composant d'infrastructure**, pas un microservice développé par l'équipe. Il fournit prêt à l'emploi : authentification multi-facteurs, OIDC/OAuth2, rôles et permissions, réinitialisation de mot de passe, audit des connexions, gestion des sessions et déconnexion centralisée.

6.3.1 Rôle de Keycloak

- **Source de vérité de l'identité** : Keycloak détient les comptes utilisateurs, les mots de passe (hashés) et les rôles applicatifs (OWNER, SECRETARY, MONITOR, CLIENT).
- **Émission des tokens JWT** signés (RS256), contenant l'identifiant utilisateur et son rôle.
- **Refresh token** avec révocation possible.
- **Traçabilité native** : chaque événement (connexion, déconnexion, échec d'authentification, changement de rôle, création de compte) est enregistré dans le journal d'événements de Keycloak.
- **Réinitialisation de mot de passe** par email et politiques de mot de passe configurables.

6.3.2 Articulation Keycloak / User Service

Une fonction synchronise les deux systèmes :

- Lorsqu'un propriétaire ou la secrétaire crée un utilisateur, le **User Service** appelle l'API Admin de Keycloak pour créer le compte d'identité, puis stocke dans sa propre base les attributs métier (forfait, progression, photo, etc.).
- Lors d'une mise à jour de profil, les attributs d'identité (email, nom) sont propagés vers Keycloak ; les attributs métier restent dans le **User Service**.
- Lors d'une suppression, l'utilisateur est désactivé dans Keycloak (pas supprimé physiquement) afin de préserver la cohérence des références d'audit (`created_by`, `updated_by`).

6.3.3 Sécurisation des microservices

Chaque microservice Spring Boot intègre l'adaptateur Keycloak (via Spring Security OAuth2 Resource Server). Le flux d'une requête authentifiée est le suivant :

1. Le client Angular obtient un JWT auprès de Keycloak (flow Authorization Code + PKCE).
2. Le client envoie ses requêtes à l'API Gateway avec l'en-tête **Authorization: Bearer <token>**.
3. L'API Gateway valide la signature du token contre la clé publique de Keycloak.
4. Le token est propagé aux microservices en aval, qui appliquent leurs règles d'autorisation locales selon le rôle.

6.4 Décomposition en microservices

Microservices proposés

La plateforme se décompose en services indépendants, chacun responsable d'un périmètre métier précis et disposant de sa propre base de données.

- **User Service** — Gestion des profils métier (propriétaire, secrétaire, moniteur, client). Synchronisation des utilisateurs avec Keycloak (création, mise à jour, désactivation). Stockage des données spécifiques à l'auto-école : forfait, progression, préférences (langue, notifications), numéro de permis.
- **Vehicle Service** — Gestion du parc automobile, entretien et affectations.
- **Booking Service** — Réservation, planification, gestion des séances.
- **Communication Service** — Messagerie temps réel (WebSocket).
- **Finance Service** — Forfaits, paiements, factures.
- **Document Service** — Stockage et gestion documentaire (intégration MinIO).
- **Notification Service** — Email, SMS, notifications push.
- **Analytics Service** — Agrégation de données, indicateurs, reporting.
- **AI Service** — Tuteur conversationnel (RAG), quiz adaptatif, examen blanc, analyse pédagogique, gestion du budget API LLM. Intègre Qdrant pour les embeddings et communique avec un fournisseur LLM externe (OpenAI ou Anthropic) via une interface abstraite.

6.5 Sécurité

- Authentification déléguée à **Keycloak** (OIDC / OAuth2), avec JWT signé en RS256 et mécanisme de Refresh Token.
- Contrôle d'accès basé sur les rôles (RBAC) au niveau de l'API Gateway et de chaque microservice, à partir des rôles portés par le token.
- Communication HTTPS de bout en bout (TLS 1.3).
- Hash des mots de passe géré par Keycloak (PBKDF2 par défaut).
- Protection contre les attaques courantes : XSS, CSRF, injection SQL, brute force (politique de verrouillage Keycloak).
- Traçabilité native via le journal d'événements Keycloak (connexions, déconnexions, échecs, changements de rôle).
- Colonnes d'audit applicatives (`created_by`, `updated_by`, `created_at`, `updated_at`) sur les entités métier, alimentées automatiquement par Spring Data JPA via l'identité extraite du JWT.
- Conformité avec les bonnes pratiques de protection des données personnelles.

7 Exigences non fonctionnelles

7.1 Performance

- Temps de réponse moyen inférieur à 500 ms pour les requêtes courantes.
- Latence des messages WebSocket inférieure à 200 ms.
- Capacité de traiter au moins 200 utilisateurs simultanés en phase initiale.
- Architecture scalable horizontalement pour absorber la croissance.

7.2 Disponibilité

- Disponibilité cible : 99,5% (hors maintenance planifiée).
- Maintenance planifiée annoncée 48h à l'avance.
- Sauvegardes quotidiennes des bases de données.

7.3 Ergonomie et accessibilité

- Interface responsive (desktop, tablette, smartphone).
- Compatibilité avec les navigateurs récents (Chrome, Firefox, Edge, Safari).
- Respect des bonnes pratiques d'accessibilité (WCAG 2.1 niveau AA).
- Interface en français (multilingue envisageable en évolution).

7.4 Maintenabilité

- Code documenté et conforme aux standards (Java : conventions Spring ; Angular : style guide officiel).
- Tests unitaires (couverture cible : 70% minimum).
- Tests d'intégration entre microservices.
- Documentation technique des API (Swagger / OpenAPI).

7.5 Portabilité et déploiement

- Conteneurisation Docker pour chaque microservice.
- Docker Compose pour l'environnement de développement.
- Possibilité de déploiement sur un cluster Kubernetes en production.

8 Stratégie d'intégration et de déploiement continu (CI/CD)

8.1 Vue d'ensemble

L'architecture microservices impose une discipline forte d'automatisation. Sans pipeline CI/CD, la coordination de neuf services indépendants devient rapidement ingérable. Le projet adopte donc une démarche **CI/CD complète** dès le démarrage, fondée sur les principes suivants :

- Chaque microservice dispose de son propre dépôt Git et de sa propre pipeline.
- Tout commit déclenche automatiquement un cycle de build, de tests et d'analyse.
- Les artefacts produits (images Docker) sont versionnés et stockés dans un registry centralisé.
- Le déploiement vers les environnements de développement et de pré-production est automatique ; la mise en production reste une action manuelle validée.
- L'infrastructure est décrite sous forme de code (Infrastructure as Code).

Outils retenus

Plateforme CI/CD : GitLab CI (offre gratuite GitLab.com).

Structure du code : multi-dépôts (un dépôt par microservice + un dépôt d'infrastructure).

Cible de déploiement : Cluster Kubernetes managé en cloud (AWS EKS, Azure AKS ou GCP GKE selon arbitrage final).

8.2 Structure des dépôts et conventions

8.2.1 Organisation multi-dépôts

- Un dépôt Git par microservice (`user-service`, `vehicle-service`, `booking-service`, etc.).
- Un dépôt dédié au frontend Angular.
- Un dépôt central `infra` contenant les manifestes Kubernetes globaux, les configurations Keycloak, les fichiers Helm, le `docker-compose.yml` de développement local, et le fichier de versions consolidées.

- Un dépôt **ci-templates** contenant les templates de pipelines partagés entre microservices.

8.2.2 Stratégie de branches

Le projet suit un modèle Gitflow simplifié :

- **main** — branche de production, protégée.
- **develop** — branche d'intégration, environnement de staging.
- **feature/xxx** — développement d'une fonctionnalité.
- **hotfix/xxx** — correctifs urgents en production.

8.2.3 Versionnement des artefacts

Chaque image Docker est taguée selon trois schémas complémentaires :

- **<service>:<commit-sha>** — tag immuable, traçabilité absolue.
- **<service>:develop** — dernière version stable de staging.
- **<service>:latest** — dernière version stable de production (sur **main**).

8.3 Pipeline d'intégration continue (CI)

Chaque microservice exécute le même squelette de pipeline, défini dans un template partagé pour éviter la duplication.

Étape	Outils	Action
Build	Maven, Node.js	Compilation du code, résolution des dépendances.
Tests unitaires	JUnit 5, Mockito, Jasmine	Exécution des tests unitaires avec rapport de couverture.
Analyse qualité	SonarCloud	Détection des bugs, code smells, dette technique. Seuil de couverture minimum : 70%.

Étape	Outils	Action
Standards de code	Checkstyle, SpotBugs, ESLint, Prettier	Application des conventions de style et des bonnes pratiques.
Scan des dépendances	OWASP Dependency-Check	Identification des bibliothèques tierces vulnérables.
Build image Docker	Docker, Dockerfile multi-stage	Construction d'une image légère prête à déployer.
Scan image Docker	Trivy	Détection des vulnérabilités dans l'image (CVE, secrets, mauvaises configurations).
Publication	GitLab Container Registry	Stockage de l'image versionnée.

8.4 Pipeline de déploiement continu (CD)

8.4.1 Environnements cibles

Trois environnements isolés en termes de namespace Kubernetes et de bases de données :

- **Développement** (auto-ecole-dev) — déploiement automatique sur push de la branche `develop`. Données de test.
- **Pré-production / Staging** (auto-ecole-staging) — déploiement automatique après validation de la CI sur `develop`. Données représentatives, examens blancs.
- **Production** (auto-ecole-prod) — déploiement **manuel** (validation explicite via bouton dans GitLab) sur push de la branche `main`. Données réelles.

8.4.2 Stratégie de déploiement

Le déploiement utilise la stratégie **Rolling Update** native de Kubernetes : les pods sont remplacés progressivement, garantissant une disponibilité continue. Chaque déploiement valide les conditions suivantes avant de passer au pod suivant :

- Sondes `readinessProbe` et `livenessProbe` exposées par chaque service via Spring Boot Actuator.
- Rollback automatique en cas d'échec du déploiement (`kubectl rollout undo`).
- Tests de fumée (smoke tests) exécutés après déploiement pour valider le bon fonctionnement.

8.4.3 Gestion des migrations de base de données

Les évolutions de schéma de base de données sont versionnées avec **Flyway** (ou Liquibase). Les scripts de migration sont exécutés automatiquement au démarrage de chaque microservice, garantissant la cohérence entre le code et le schéma de données.

8.5 Gestion des secrets et configuration

- Aucun secret (mot de passe, clé API LLM, credentials Keycloak admin) n'est versionné dans le code.
- Les secrets sont stockés dans **GitLab CI/CD Variables** (masquées, protégées) pour les pipelines.
- En cluster, les secrets sont gérés par **Kubernetes Secrets**, idéalement chiffrés au repos (Sealed Secrets, External Secrets Operator ou intégration avec un coffre-fort cloud type AWS Secrets Manager).
- Les configurations non sensibles utilisent des **ConfigMaps** Kubernetes.

8.6 Orchestration multi-dépôts

L'architecture multi-dépôts soulève la question de la coordination des déploiements globaux. Deux mécanismes complémentaires sont mis en place :

- **Fichier de versions centralisé** (`versions.yaml` dans le dépôt `infra`) listant la version actuellement déployée de chaque microservice en production. Mis à jour automatiquement par la pipeline de chaque service après déploiement réussi.
- **Pipeline triggers inter-projets** : la pipeline d'un microservice peut déclencher une pipeline en aval dans le dépôt `infra` pour orchestrer un déploiement coordonné.

8.7 Phases de mise en place de la CI/CD

La construction de la chaîne CI/CD se fait par incréments, en commençant par un service pilote (typiquement le `user-service`) puis en généralisant.

Phase	Objectif	Livrables
Phase 0	Préparation : choix du cloud, conventions de branches, conventions de tags, provisioning du registry et du cluster Kubernetes.	Cluster Kubernetes opérationnel, dépôts Git initialisés, conventions documentées.
Phase 1	CI basique : build et tests unitaires par microservice.	Pipeline <code>.gitlab-ci.yml</code> fonctionnel sur chaque dépôt avec stages build et test.
Phase 2	CI complète : analyse qualité (SonarCloud), scan de dépendances (OWASP), build d'image Docker, scan d'image (Trivy), publication au registry.	Image Docker versionnée et scannée pour chaque commit.
Phase 3	CD automatique : déploiement automatique sur <code>dev</code> et <code>staging</code> , manuel sur <code>prod</code> . Manifestes Kubernetes complets.	Déploiement reproductible sur les trois environnements.
Phase 4	Orchestration multi-dépôts : dépôt <code>infra</code> , fichier <code>versions.yaml</code> , triggers inter-projets, templates partagés.	Cohérence des déploiements multi-services.
Phase 5	Observabilité : monitoring (Prometheus + Grafana), logs centralisés (ELK ou Loki), tracing distribué (Jaeger), tests d'intégration E2E (Postman/Newman, Cypress).	Vue temps réel de l'état du système, capacité de diagnostic.

8.8 Indicateurs de qualité visés

- Couverture de tests unitaires : 70% minimum par microservice.
- Durée d'une pipeline complète : moins de 10 minutes par microservice.
- Aucune vulnérabilité critique ou élevée non corrigée dans les images Docker publiées.
- Temps de déploiement (de **develop** à **staging**) : moins de 5 minutes.
- Taux d'échec des déploiements en production : moins de 5%.

9 Planning prévisionnel

Le projet est organisé selon une méthodologie agile (Scrum) avec des sprints de deux semaines. Le planning prévisionnel ci-dessous est indicatif et pourra être ajusté.

Phase	Description	Livrables
Phase 1	Étude, conception, maquettage	Cahier des charges, maquettes UI, modèle de données
Phase 2	Mise en place de l'infrastructure	Configuration Docker, Gateway, Eureka, Keycloak (realms, rôles, clients)
Phase 3	Développement des services métier (User, Vehicle, Booking)	Backend opérationnel pour la planification
Phase 4	Développement des services Finance & Document	Forfaits, paiements, gestion documentaire
Phase 5	Communication temps réel	WebSocket, messagerie, notifications
Phase 6	Module IA — Tuteur RAG, quiz adaptatif, examen blanc	ai-service, Qdrant, ingestion documentaire, intégration LLM
Phase 7	Frontend Angular	Interfaces des 4 rôles
Phase 8	Analytics, dashboard pédagogique IA, dashboard coûts	Indicateurs, exports, reporting, vue propriétaire
Phase 9	Tests, recette, déploiement	Application prête à la mise en production

10 Contraintes et livrables

10.1 Contraintes

- Respect des technologies imposées (Java/Spring Boot, Angular, PostgreSQL).
- Architecture obligatoirement microservices.
- Communication temps réel via WebSocket.
- Respect des règles de protection des données personnelles.
- Documentation complète à fournir avec le code source.

10.2 Livrables attendus

- Code source complet (backend, frontend, scripts de déploiement).
- Documentation technique (architecture, API, base de données).
- Documentation utilisateur (manuels par rôle).
- Jeux de tests (unitaires et d'intégration).
- Environnement Docker prêt à l'emploi.
- Rapport de recette et procès-verbal de réception.

11 Évolutions envisagées

Les fonctionnalités suivantes sont identifiées comme évolutions potentielles pour des versions ultérieures :

- Application mobile native (iOS / Android).
- Paiement en ligne intégré (Stripe, gateway local).
- Module e-learning pour l'apprentissage du code en ligne.
- Géolocalisation des points de rendez-vous et suivi du véhicule.
- Analyse d'image par IA (l'élève photographie un panneau, l'IA explique la règle associée).
- Migration vers un LLM auto-hébergé (LLaMA, Mistral) pour réduire la dépendance à un fournisseur externe et améliorer la confidentialité.
- Gamification de l'apprentissage : badges, séries (streaks), niveaux, classements opt-in.
- Module multi-établissements pour les réseaux d'auto-écoles.
- Application multilingue étendue (anglais, autres langues régionales).

12 Glossaire

Terme	Définition
API	Application Programming Interface — interface permettant à des applications de communiquer entre elles.
JWT	JSON Web Token, format de jeton sécurisé utilisé pour l'authentification.
Microservice	Architecture logicielle décomposant une application en services indépendants.
RBAC	Role-Based Access Control — modèle de gestion des permissions basé sur les rôles.
REST	Representational State Transfer — style d'architecture pour les API web.
WebSocket	Protocole de communication bidirectionnelle en temps réel.
Forfait	Pack d'heures de conduite et/ou de séances de code vendu à l'élève.
Gateway	Point d'entrée unique vers les microservices.
Eureka	Service de découverte de microservices de la suite Spring Cloud.
MinIO	Système de stockage objet compatible avec Amazon S3.
LLM	Large Language Model — grand modèle de langage capable de comprendre et générer du texte (ex : GPT-4o, Claude).
RAG	Retrieval-Augmented Generation — technique combinant un LLM avec une base documentaire pour produire des réponses ancrées dans des sources fiables.
Embedding	Représentation vectorielle d'un texte permettant de mesurer sa similarité avec d'autres textes.

Terme	Définition
Base vectorielle	Base de données spécialisée dans la recherche de similarité entre vecteurs (Qdrant, pgvector).
Hallucination	Production par un LLM d'une information fausse ou inventée, présentée avec assurance.
Chunking	Découpage d'un document en passages courts pour l'indexation et la recherche.
Streaming (LLM)	Diffusion progressive de la réponse du modèle, token par token, sans attendre la fin de génération.
Keycloak	Solution open-source de gestion d'identité et d'accès (IAM), utilisée pour l'authentification, les rôles et l'émission des tokens.
IAM	Identity and Access Management — gestion centralisée des identités et des autorisations.
OIDC	OpenID Connect — protocole d'authentification construit sur OAuth 2.0.
PKCE	Proof Key for Code Exchange — extension d'OAuth 2.0 sécurisant l'échange de code pour les clients publics (SPA, mobile).
CI/CD	Continuous Integration / Continuous Delivery — pratiques d'automatisation du build, des tests et du déploiement.
GitOps	Pratique consistant à utiliser un dépôt Git comme source de vérité de l'infrastructure et des déploiements.
